

Neural Architecture Search (Part II)

서울대학교 컴퓨터공학부 이영기

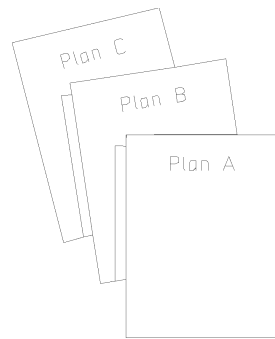
Overview

- Objective

- Understand challenges in traditional ML pipeline
- Learn various Hyperparameter Optimization (HPO) techniques

- Content

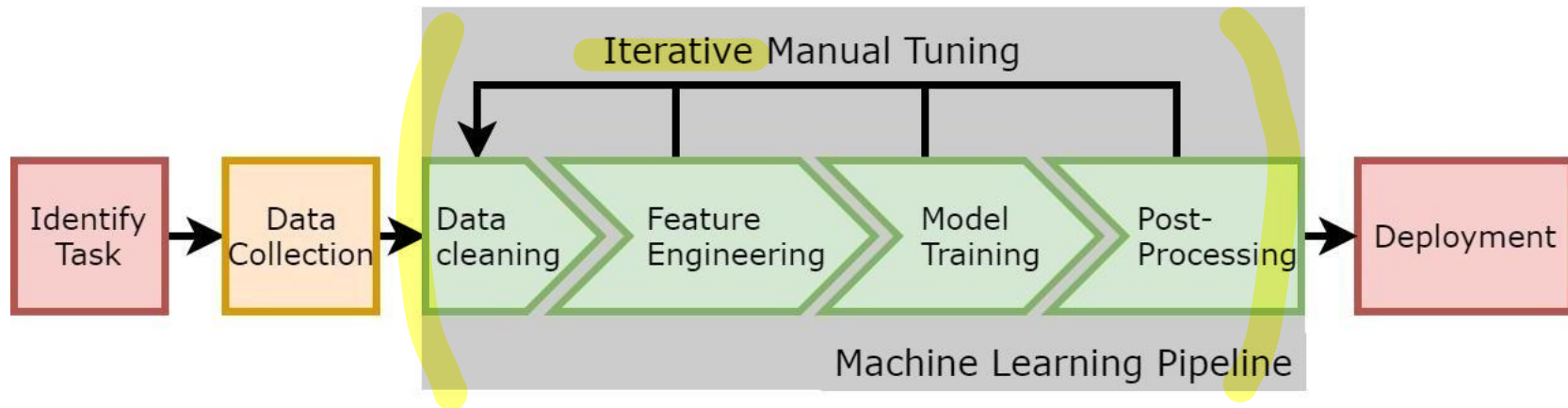
- Introduction to AutoML
- Hyperparameter optimization techniques
 - Blackbox optimization(Grid search, Random search, Bayesian optimization, Tree-structured Parzen Estimator)
 - Multi-fidelity optimization(Successive halving, Hyperband, Bayesian optimization + Hyperband)



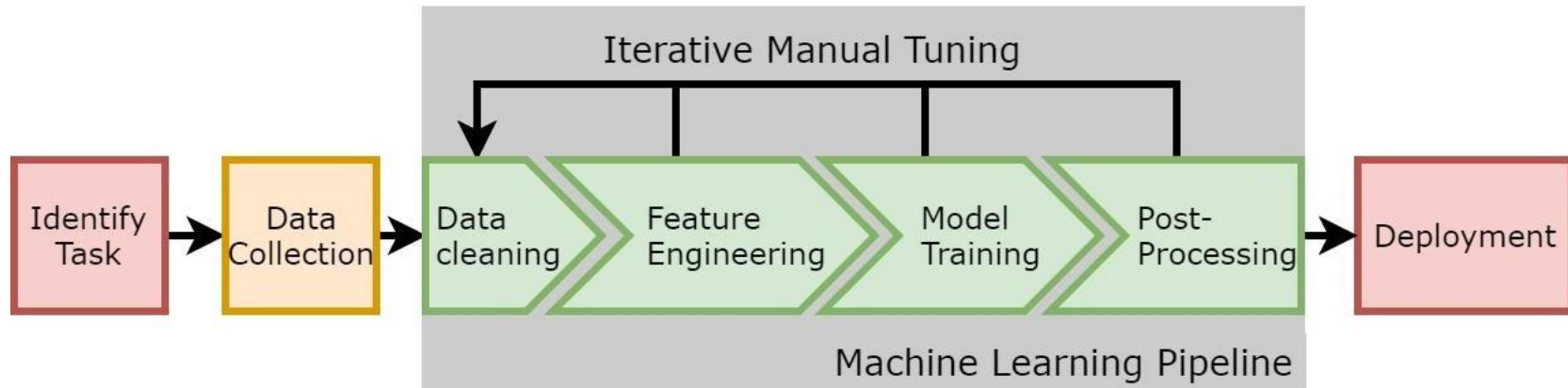
Traditional Machine Learning Pipeline

- Building an ML model is an iterative, complex, and time-consuming process
 - Clean & preprocess the data
 - Select and engineer better features
 - Select an algorithm
 - Set the model parameters
 - ...

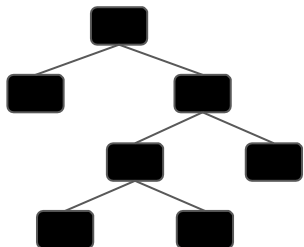
어쩔 수 없어 여러 함수를 해봐야한다



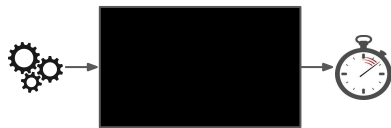
Challenges in Designing ML Pipelines



Complex Search Space



Black-Box Problem



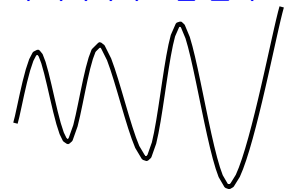
Expensive Evaluations

학습을 해야하니까

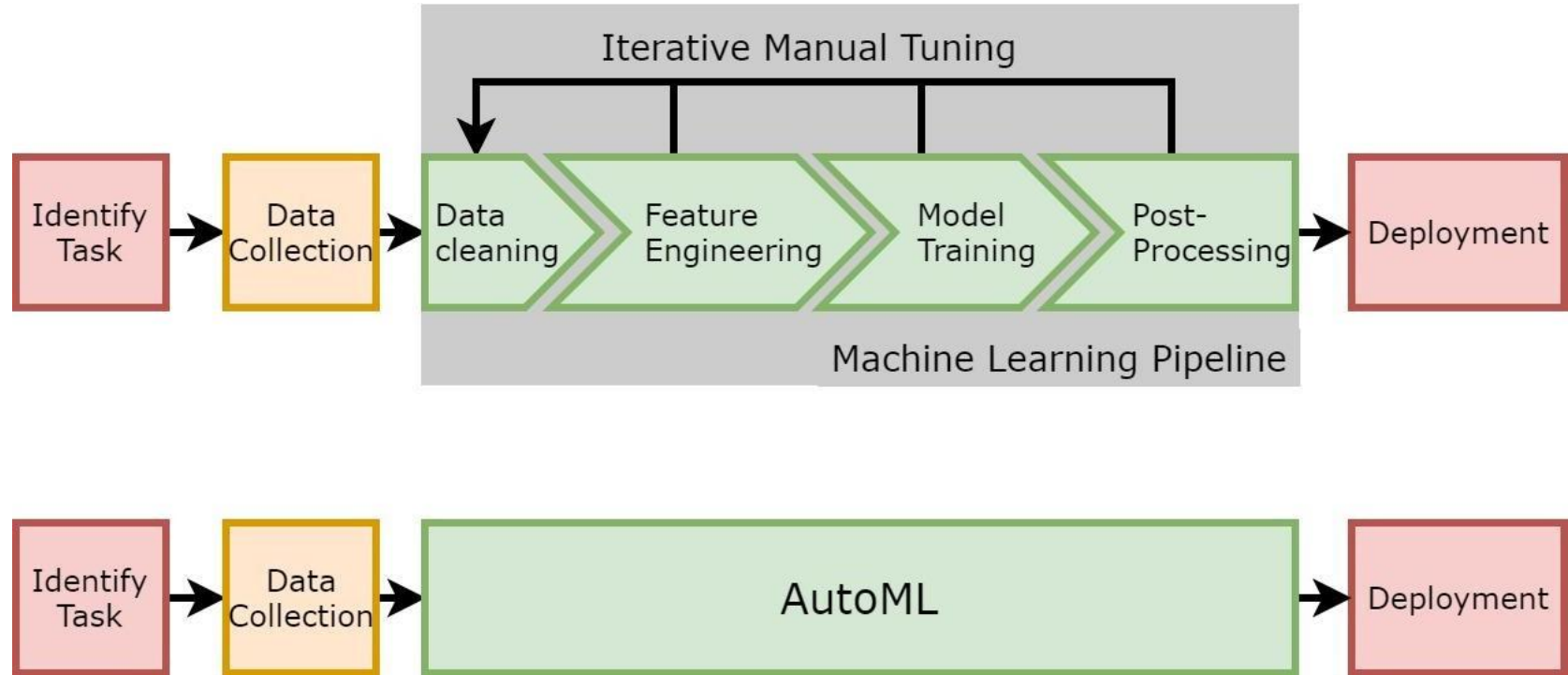


Noise on observations

새 데이터에도 잘 될지 모름

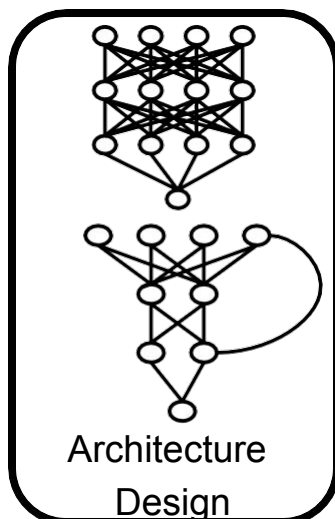
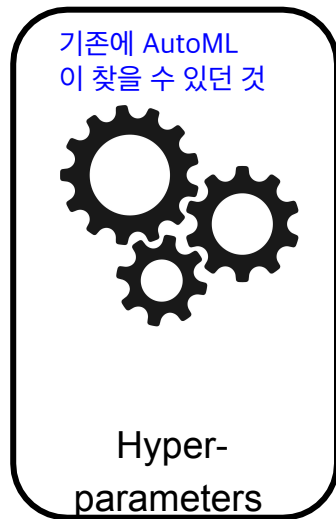


From Manual to Automated ML



Design Decisions by AutoML

요즘에는 이런 것들도 다 해준다



preprocessor	#λ
extreml. rand. trees prepr.	5
fast ICA	4
feature agglomeration	4
kernel PCA	5
rand. kitchen sinks	2
linear SVM prepr.	3
no preprocessing	-
nystroem sampler	5
PCA	2
polynomial	3
random trees embed.	4
select percentile	2
select rates	3
one-hot encoding	2
imputation	1
balancing	1
rescaling	1

Pre-
processing

classifier	#λ
AdaBoost (AB)	4
Bernoulli naïve Bayes	2
decision tree (DT)	4
extreml. rand. trees	5
Gaussian naïve Bayes	-
gradient boosting (GB)	6
kNN	3
LDA	4
linear SVM	4
kernel SVM	7
multinomial naïve Bayes	2
passive aggressive	3
QDA	2
random forest (RF)	5
Linear Class. (SGD)	10

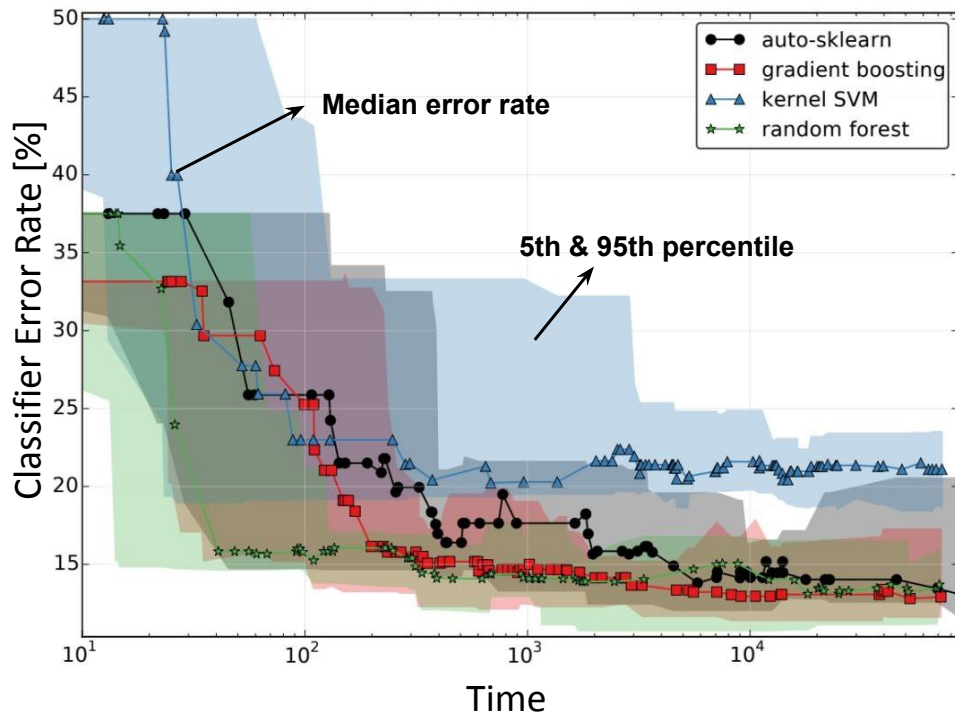
Algorithms

ex. SVM 에서

- 커널 (차원 변환)
- maximal margin classifier에서 $\text{margin} = C$

Importance of Design Decisions in ML

- Each decision is critical to overall performance
 - E.g. Classifiers performance compared to auto-sklearn over time in one OpenML dataset

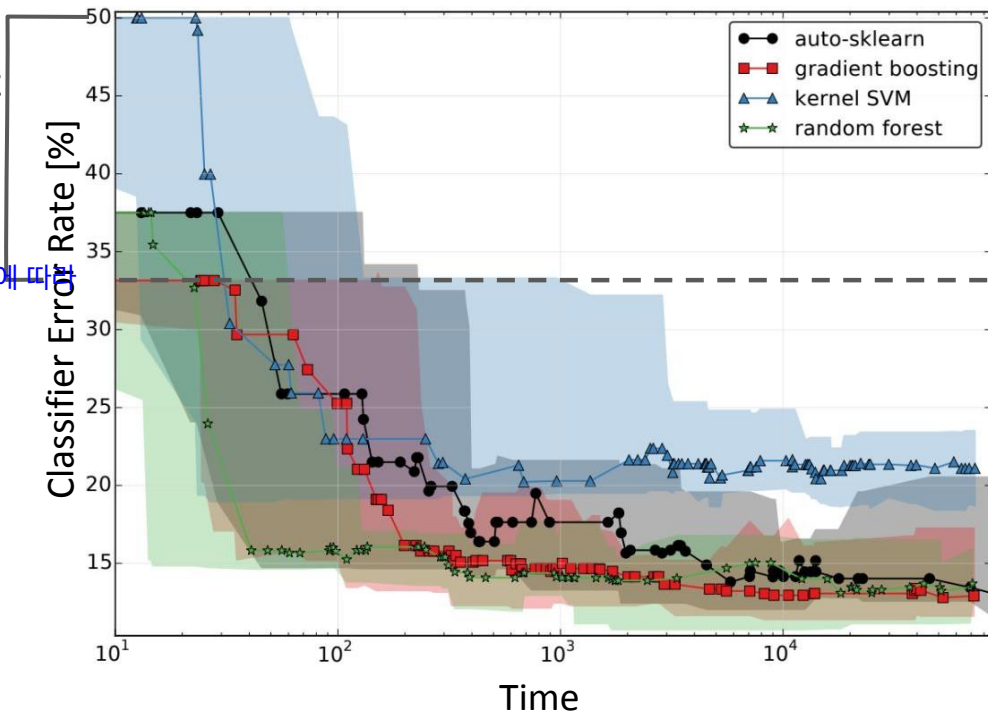


Importance of Design Decisions in ML

- Each decision is critical to overall performance
 - E.g. Classifiers performance compared to auto-sklearn over time in one OpenML dataset

Choosing the gradient
boosting algorithm
instead of kernel SVM
→ 17% improv.

어떤 알고리즘을 선택하느냐에 따라
performance 가 차이난다



시간을 많이 쓰면 더 많은 hyperparameter를
써봐서 더 좋은 걸 선택할수있다

Optimizing

hyperparameters

in gradient boosting

→ 20% improv.

개별 알고리즘의 hyperparam도
중요하다

Overview

- Objective

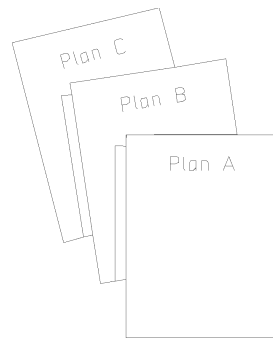
- Understand challenges in traditional ML pipeline
- Learn various Hyperparameter Optimization (HPO) techniques

- Content

- ~~Introduction to AutoML~~

- **Hyperparameter optimization techniques**

- Blackbox optimization(Grid search, Random search, Bayesian optimization, Tree-structured Parzen Estimator)
 - Multi-fidelity optimization(Successive halving, Hyperband, Bayesian optimization + Hyperband)



Hyperparameter Optimization

Definition: Hyperparameter Optimization (HPO)

Let

- λ be the hyperparameters of a ML algorithm A with domain Λ ,
- $\mathcal{L}(A_\lambda, D_{train}, D_{valid})$ denote the loss of A , using hyperparameters λ trained on D_{train} and evaluated on D_{valid} .

The **hyperparameter optimization (HPO)** problem is to find a **hyperparameter configuration λ^*** that minimizes this loss:

$$\lambda^* \in \arg \min_{\lambda \in \Lambda} \mathcal{L}(\underbrace{A_\lambda}_{\text{알고리즘}}, D_{train}, \underbrace{D_{valid}}_{\text{가용3}})$$

test / train 나누는 데 주의:

- 데이터 처리(clean, 전처리) 하는 사람은 val data를 고려하지 않음
- 근데 데이터 모델링하는 사람은 고려함
- 전처리에서 데이터를 다 보는 것은 val 을 이미 보고 처리하는 거라서 반칙하게 됨

Types of Hyperparameters (1/2)

- Continuous
 - Regularization parameter C in SVM
 - Shrinkage parameter λ in boosting
- Integer
 - Number of trees in boosting and random forest
- Categorical
 - Algorithm (SVM, RF, NN, ...)
 - SVM kernel (Polynomial, RBF, ...)
 - Activation function (ReLU, Leaky ReLU, ...)

Types of Hyperparameters (2/2)

- Conditional: hyperparameters B are only active if other hyperparameters A are set a certain way 알고리즘을 고르면 거기에 맞는 hyperparameter 이 따로 있다..
 - Example 1:
 - A = choice of optimizer (Adam or SGD)
 - B = Adam's second momentum hyperparameter (only active if A=Adam)
 - Example 2:
 - A = type of layer k (convolution, max pooling, fully connected, ...)
 - B = conv. kernel size of that layer (only active if A = convolution)
 - Example 3:
 - A = choice of classifier (RF or SVM)
 - B = SVM's kernel parameter (only active if A = SVM)

AutoML as Hyperparameter Optimization

Definition: Combined Algorithm Selection and Hyperparameter Optimization (CASH)

Let

- $\mathcal{A} = \{A^{(1)}, \dots, A^{(n)}\}$ be a set of algorithms
- $\Lambda^{(i)}$ denote the hyperparameter space of $A^{(i)}$, for $i = 1, \dots, n$
- $\mathcal{L}(A_{\lambda}^{(i)}, D_{train}, D_{valid})$ denote the loss of $A^{(i)}$, using $\lambda \in \Lambda^{(i)}$ trained on D_{train} and evaluated on D_{valid} .

The Combined Algorithm Selection and Hyperparameter Optimization (CASH) problem is to find a combination of algorithm $A^* = A^{(i)}$ and hyperparameter configuration $\lambda^* \in \Lambda^{(i)}$ that minimizes this loss:

$$A_{\lambda^*}^* \in \arg \min_{A^{(i)} \in \mathcal{A}, \lambda \in \Lambda^{(i)}} \mathcal{L}(A_{\lambda}^{(i)}, D_{train}, D_{valid})$$

아까는 알고리즘이 고정이었으나, 여기서는 알고리즘 또한 변수임
알고리즘도 결정을 해야함

→ Simply a HPO problem with a top-level hyperparameter (choice of algorithm) that all other hyperparameters are conditional on.

Overview

- Objective

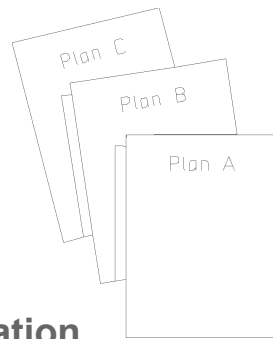
- Understand challenges in traditional ML pipeline
- Learn various Hyperparameter Optimization (HPO) techniques

- Content

- ~~Introduction to AutoML~~

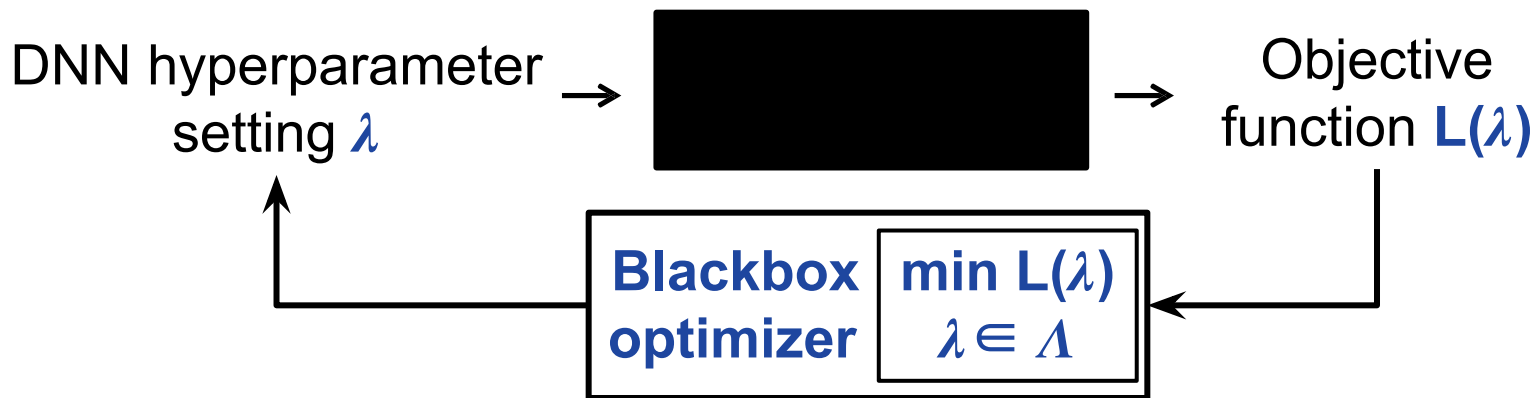
- ~~Hyperparameter optimization techniques~~

- **Blackbox optimization(Grid search, Random search, Bayesian optimization, Tree-structured Parzen Estimator)**
 - Multi-fidelity optimization(Successive halving, Hyperband, Bayesian optimization + Hyperband)



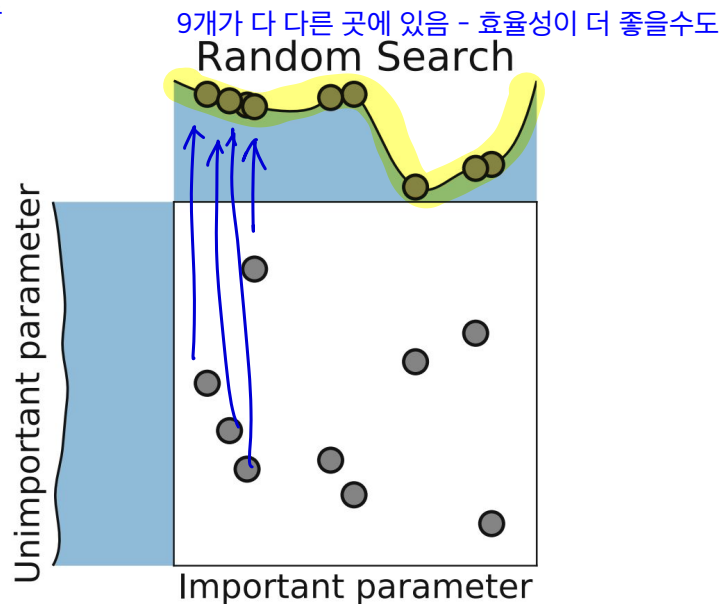
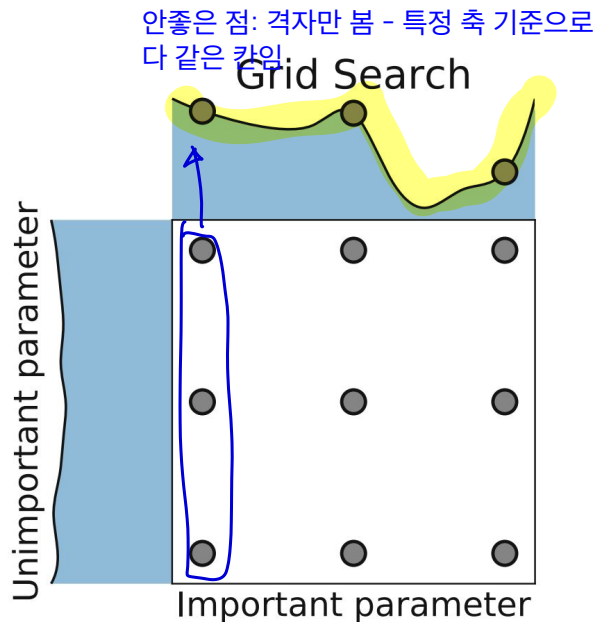
Blackbox Hyperparameter Optimization

- The blackbox function is expensive to evaluate
→ Can't sample too often!



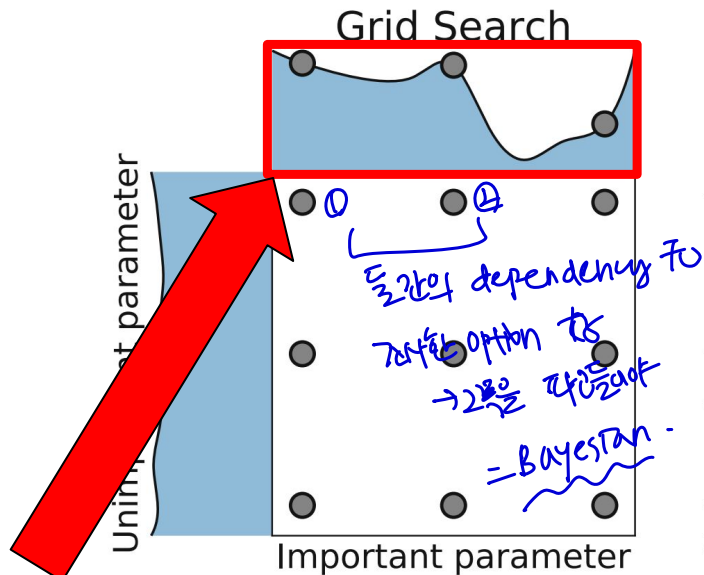
Blackbox Optimization: Grid and Random Search

- Grid search: required number of samples grows exponentially
- Random search: handles unimportant dimensions better

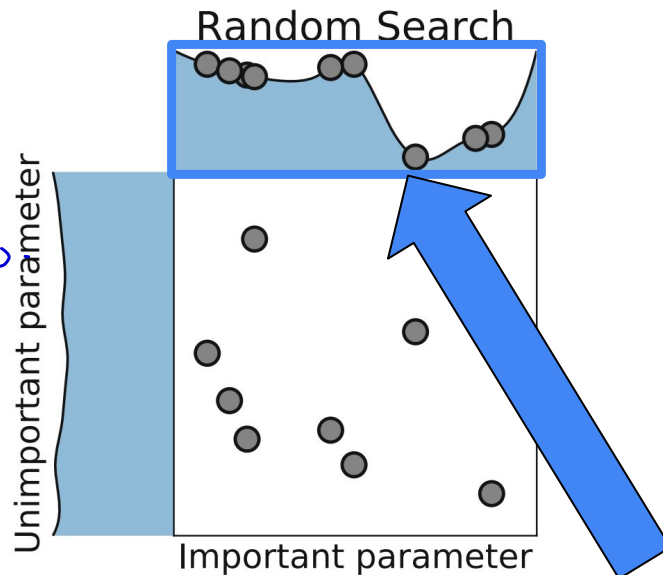


Blackbox Optimization: Grid and Random Search

- Grid search: required number of samples grows exponentially
- Random search: handles unimportant dimensions better



Only 3 useful evaluations



9 useful evaluations 17

Bayesian Optimization

Random
Grid

vs

Bayesian

조건부 확률 — conditional to what?

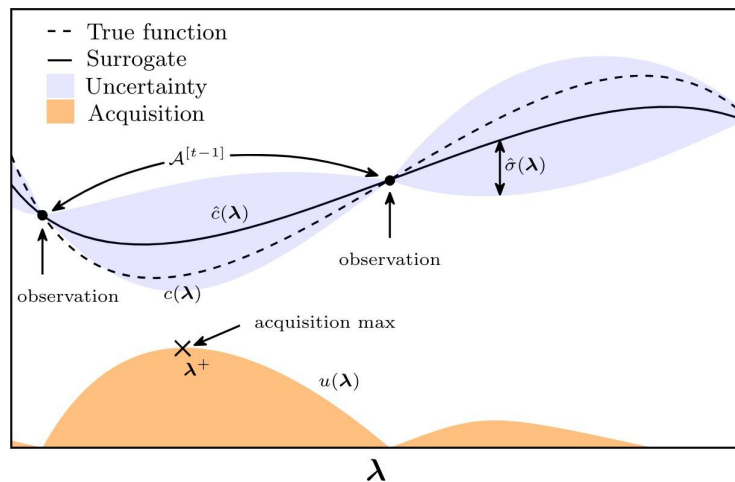
- Limitation of previous techniques: completely uninformed and does not learn about the search space
- If we have already evaluated a set of candidates, can't we do better than random selection?

use conditional probability.

=> **Predict promising regions based on history of observations**

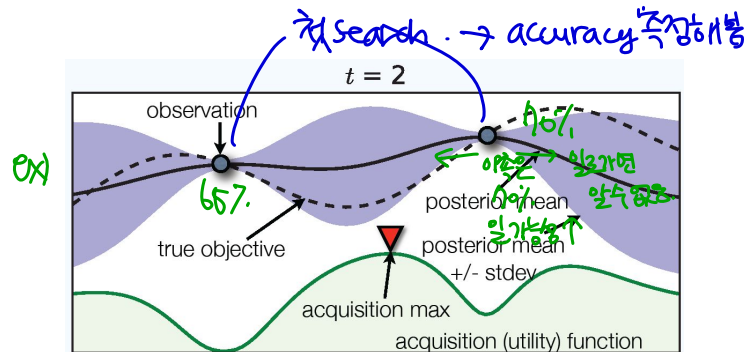
Bayesian Optimization

- Update the surrogate model sequentially with new observations
 - Use the acquisition function to select new promising configurations to evaluate
 - Surrogate Model
 - Probabilistic modeling of $L(\lambda) \sim (\hat{L}(\lambda), \hat{\sigma}(\lambda))$ with posterior mean $\hat{L}(\lambda)$ and uncertainty $\hat{\sigma}(\lambda)$
E.g. Gaussian Processes
 - Acquisition Function
 - Balance exploration (high $\hat{\sigma}$) vs. exploitation (low $\hat{L}$)
 - E.g. Expected improvement (EI):
 $u(\lambda) = E[\max\{L_{\min} - L(\lambda), 0\}]$, where $L_{\min}$ is lowest value from observations
-



Bayesian Optimization

1. Start with a few random hyperparameter configurations *처음에는 random search*
2. Fit a surrogate model to observations *↳ accuracy를 추정하는*
3. Compute the acquisition function and select new configurations accordingly
4. Evaluate new points and get observations
5. Repeat 2-4



→ 그 다음엔 어디해볼것? $\rightarrow$ acquisition func
 → 일기함 $\rightarrow$ 대? 머릿속에 추정해볼거 같은거
 (정확성, confidence는 보라영역)

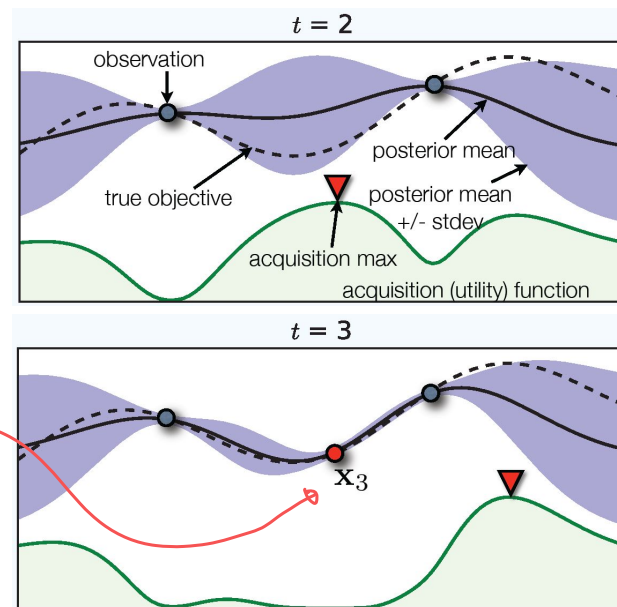
참고) uncertainty 가 낮은쪽은

한번만큼 방문해야함 = exploration

(대? 모르는거확인함)

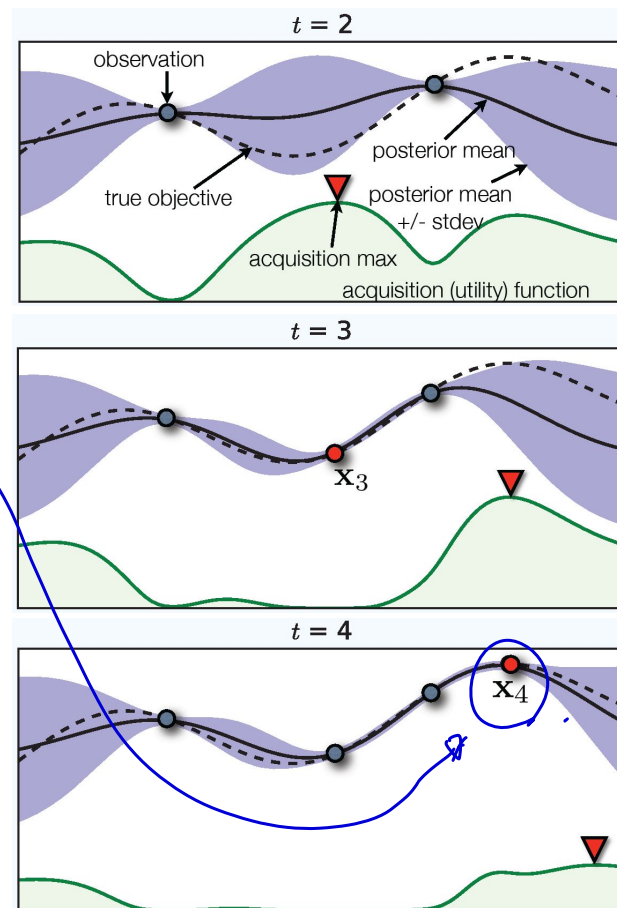
Bayesian Optimization

1. Start with a few random hyperparameter configurations
2. Fit a surrogate model to observations
3. Compute the acquisition function and select new configurations accordingly
4. Evaluate new points and get observations
5. Repeat 2-4



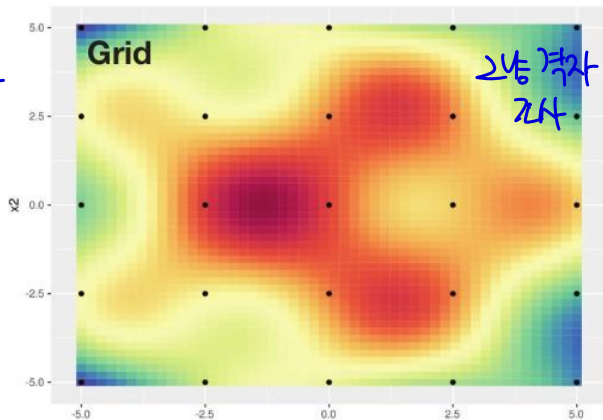
Bayesian Optimization

1. Start with a few random hyperparameter configurations
2. Fit a surrogate model to observations
3. Compute the acquisition function and select new configurations accordingly
4. Evaluate new points and get observations
5. Repeat 2-4



Bayesian Optimization

실제 accuracy가
높은 지점 = red



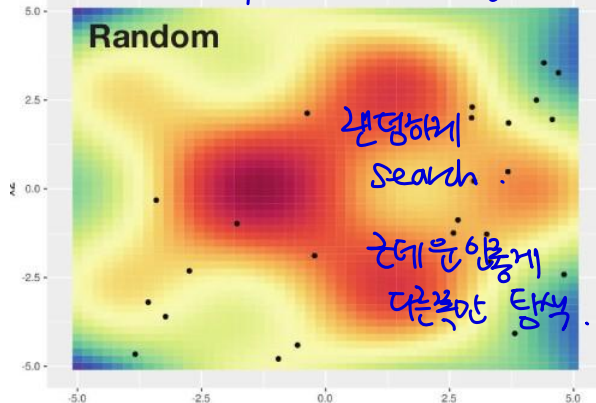
단점 ① sequential 하다.

→ parallelize 어려움, 느림.

→ random 시작 지점 바꿔보기 / 영역 쪼개보기.. 그리고 HW 나눠서.

→ 근데 그럼 Bayesian 의 원리에 좀 반한다.

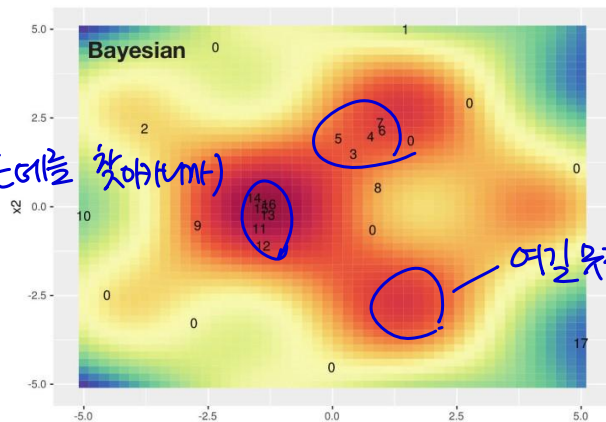
② surrogate model 이
좋아야 함 - 어려움



이걸 잘만들자
= TPE.

→ 집중 조사함

(탐색을 높은데를 찾아서)



여기 무장 → exploration 안했어..

Tree-structured Parzen Estimator (TPE)

- Problems for standard Gaussian Process approach:
 - Not scalable to high dimensional hyperparameter spaces
 - Unfit for discrete spaces since GP assumes smoothness (e.g. Conditional hyperparameters)

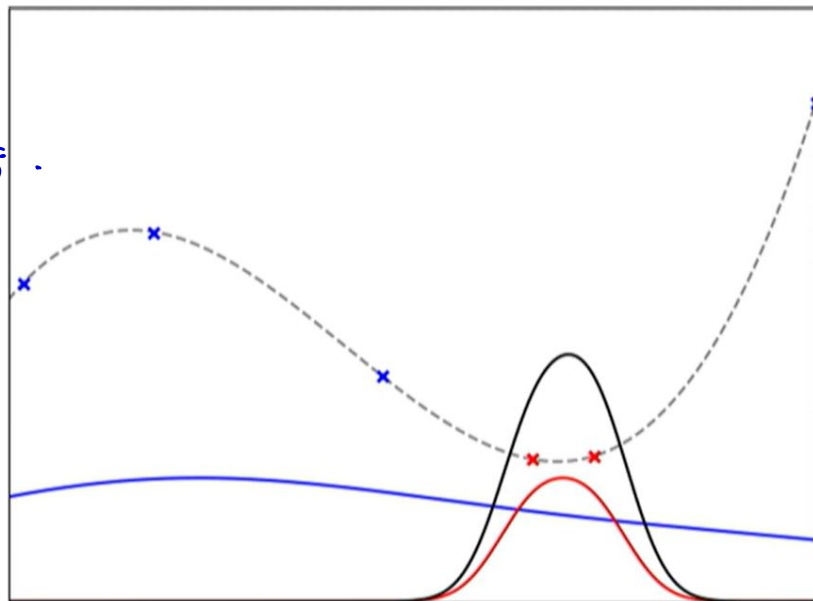
Tree-structured Parzen Estimator (TPE) 이진 방법도 있다.

- Problems for standard Gaussian Process approach:
 - Not scalable to high dimensional hyperparameter spaces
 - Unfit for discrete spaces since GP assumes smoothness (e.g. Conditional hyperparameters)

- Maintain two surrogate models**

$p(\lambda \text{ is good})$ and $p(\lambda \text{ is bad})$, Hyperparameter
중간 나눌 때 연속
rather than $p(L | \lambda)$ (이름만 X)

- $P(\lambda \text{ is good}) = P(\lambda | L \leq L^*)$
 - $P(\lambda \text{ is bad}) = P(\lambda | L > L^*)$
(L^* : split threshold)
- Acquisition function $\propto \frac{P(\lambda \text{ is good})}{P(\lambda \text{ is bad})}$
 - Equivalent to expected improvement



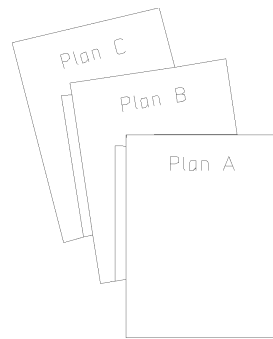
Overview

- Objective

- Understand challenges in traditional ML pipeline
- Learn various Hyperparameter Optimization (HPO) techniques

- Content

- ~~○ Introduction to AutoML~~
- ~~○ Hyperparameter optimization techniques~~
 - ~~○ Blackbox optimization (Grid search, Random search, Bayesian optimization, Tree-structured Parzen Estimator)~~
 - **Multi-fidelity optimization (Successive halving, Hyperband, Bayesian optimization + Hyperband)**



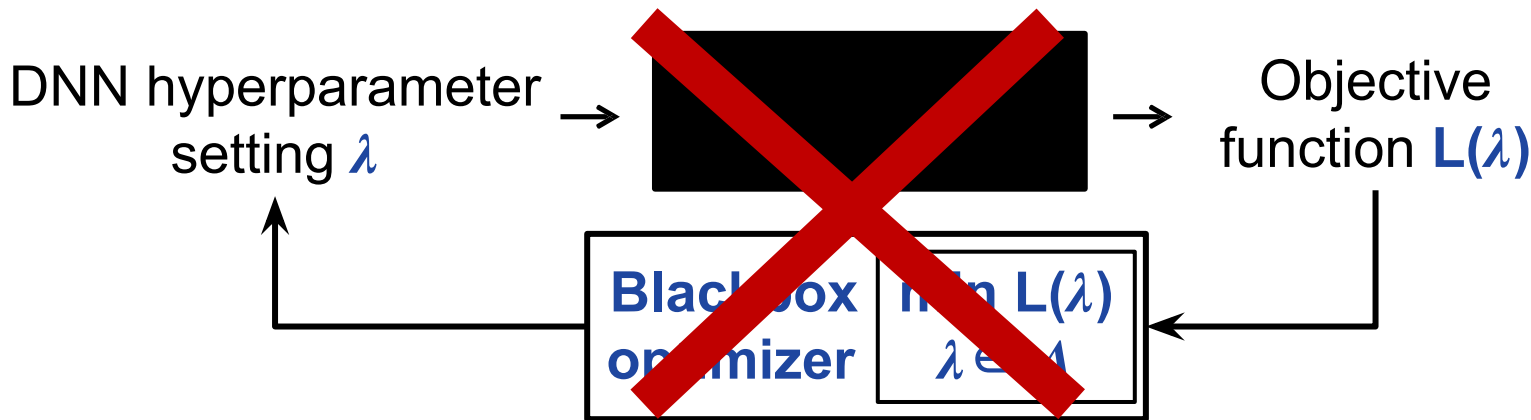
Multi-fidelity Optimization

- Massive dataset sizes and complex models make blackbox performance evaluation expensive

hyperparam 을 찾을 때 학습을 처음부터 끝까지 하는 건 costly
어떻게 빠르게 좋은 candidate 쪽으로 좁혀갈 것인가?
-> multifidelity

처음에 내가 10개의 hyperparam 중에 뭐가 좋은지 판단 시에는 작은 데이터로 해서 빠르게 버릴 영역을 버린다
5개 버렸음 - 나머지 5개에 대해서는 조금 더 많은 데이터로 해봄
그리고 또 2개 버림 - 나머지 3개에 대해서 좀더 좋은 데이터로 다시 학습

이런식으로 나아가는게 multi fidelity 이다



Too slow for DL / big data

Multi-fidelity Optimization

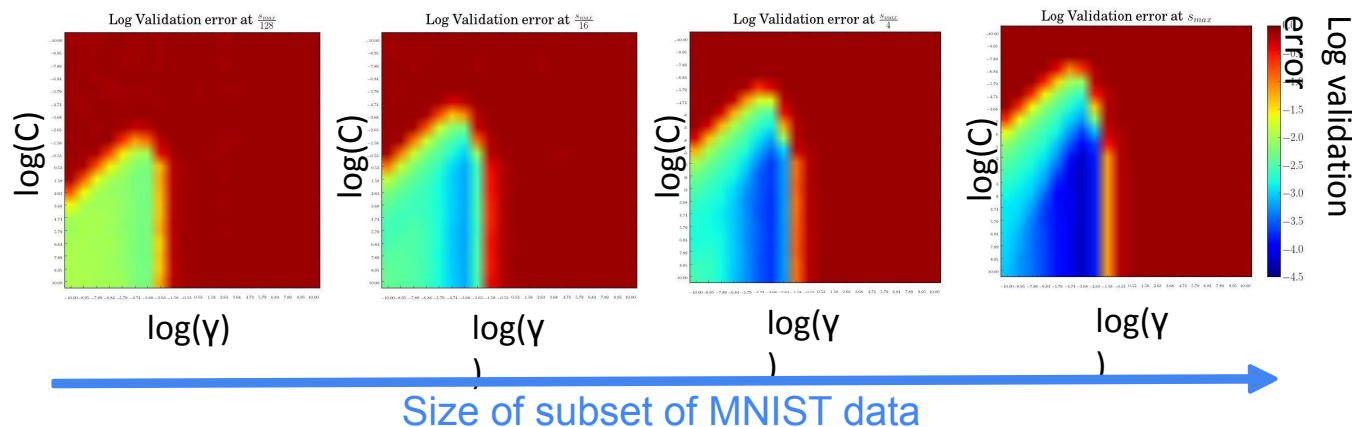
두 가지 fidelity control 방식:

1. 데이터 셋을 조금부터 시작한다 그리고 데이터 크기를 늘린다
2. epoch 을 작게 시작했다가 늘린다

- Massive dataset sizes and complex models make blackbox performance evaluation expensive
- Make use of cheap low-fidelity evaluations with a small budget, e.g.
 - Subsets of the data
 - Fewer epochs of iterative training algorithms
- These approximations introduce a tradeoff between optimization performance and runtime

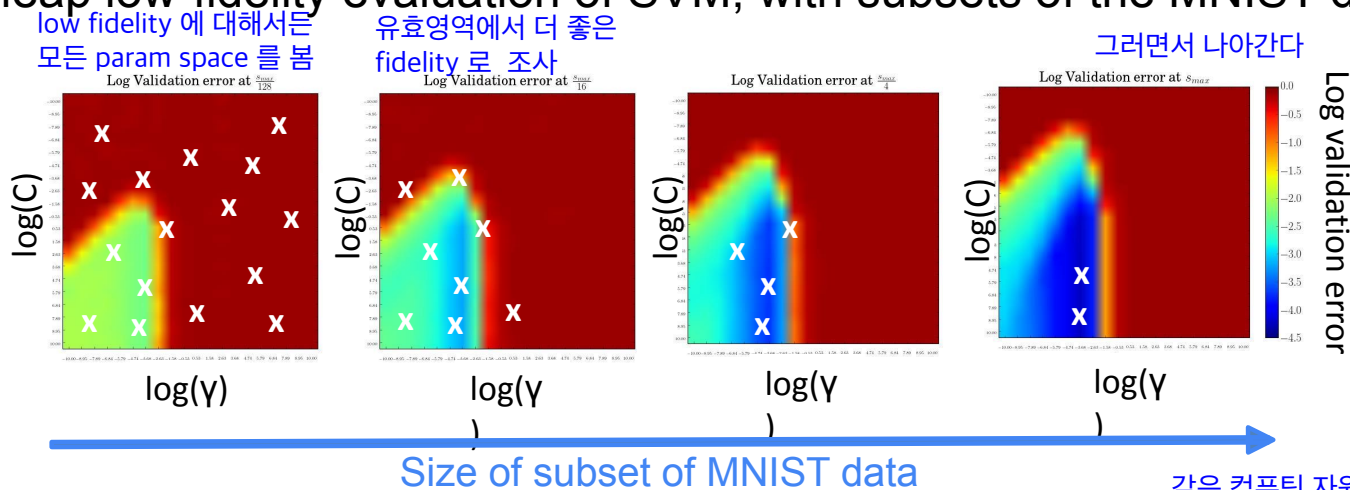
Successive Halving

- Many cheap evaluations on small subsets & few expensive evaluations on the full data
- E.g. cheap low-fidelity evaluation of SVM, with subsets of the MNIST data



Successive Halving

- Many cheap evaluations on small subsets & few expensive evaluations on the full data
- E.g. cheap low-fidelity evaluation of SVM, with subsets of the MNIST data

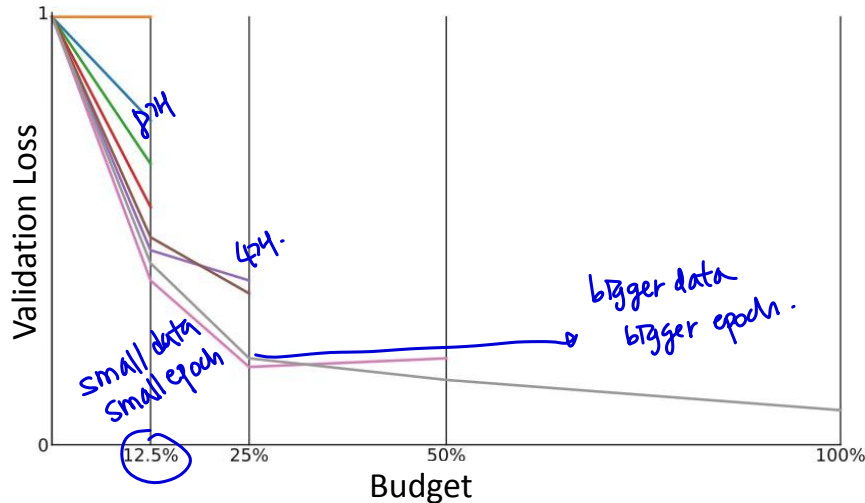


- Up to 1000x speedups

같은 컴퓨팅 자원으로
더 좋은 모델을 찾아갈수 있다

Successive Halving

- Successive halving for eight algorithms/configurations
- After evaluating all algorithms on 1/8 of the total budget, half of them are dropped and the budget given to the remaining algorithms is doubled



Successive Halving: Limitation

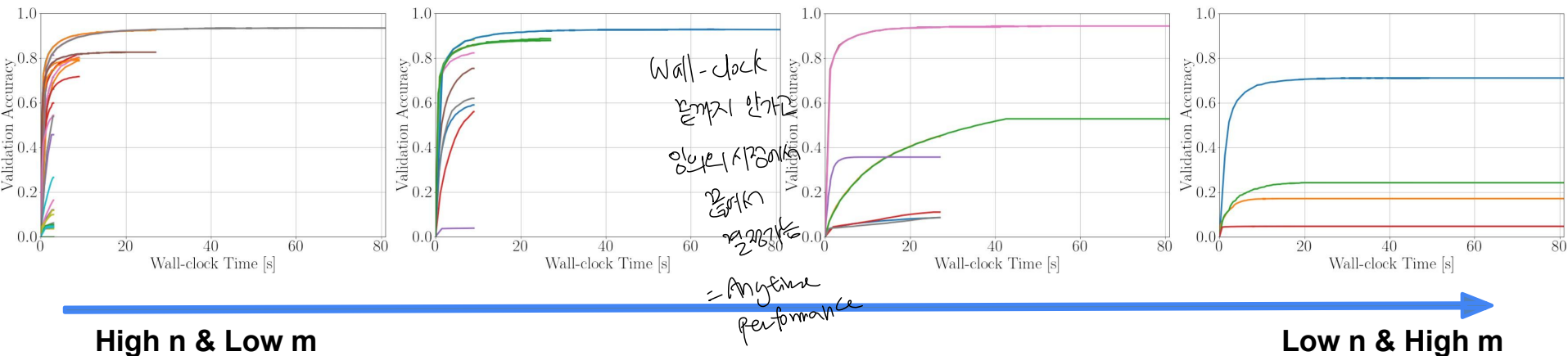
- Budget vs. number tradeoff in SHA
 - Given a total budget, the user has to decide whether:
 - Try many configurations and only assign a small budget to each
 - Try only a few and assign them a larger budget
- Assigning too small budget can result in prematurely terminating good configurations
- Assigning too large budget can result in running poor configurations too long

🕒 Budget 분배를 어떻게 할 것인가? - 이걸 정하는 것도 까다로운 부분

HyperBand

Bayesian search의 단점을 보완하고 Successive Halving의 장점을 가져오는 방식..

- Trying various budget-number combinations
 - Repeat SHA with different initial number of configurations n and start budgets m
 - Randomly select the initial configurations for each iteration
 - Return configuration with the smallest intermediate loss seen so far



BOHB: Bayesian Optimization and HyperBand

- Advantage of Bayesian optimization
 - Strong final performance
- Advantage of HyperBand
 - Strong anytime performance
 - Easily parallelizable
 - Scalable

BOHB: Bayesian Optimization and HyperBand

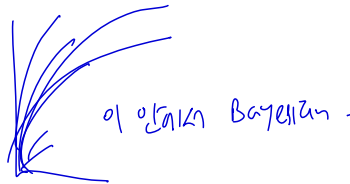
- Advantage of Bayesian optimization

- Strong final performance $\ominus$ sequential $\rightarrow$ 1/2

기본적으로는 hyperband 인데
그 안에서 search 하는 걸 bayesian 으로

- Advantage of HyperBand

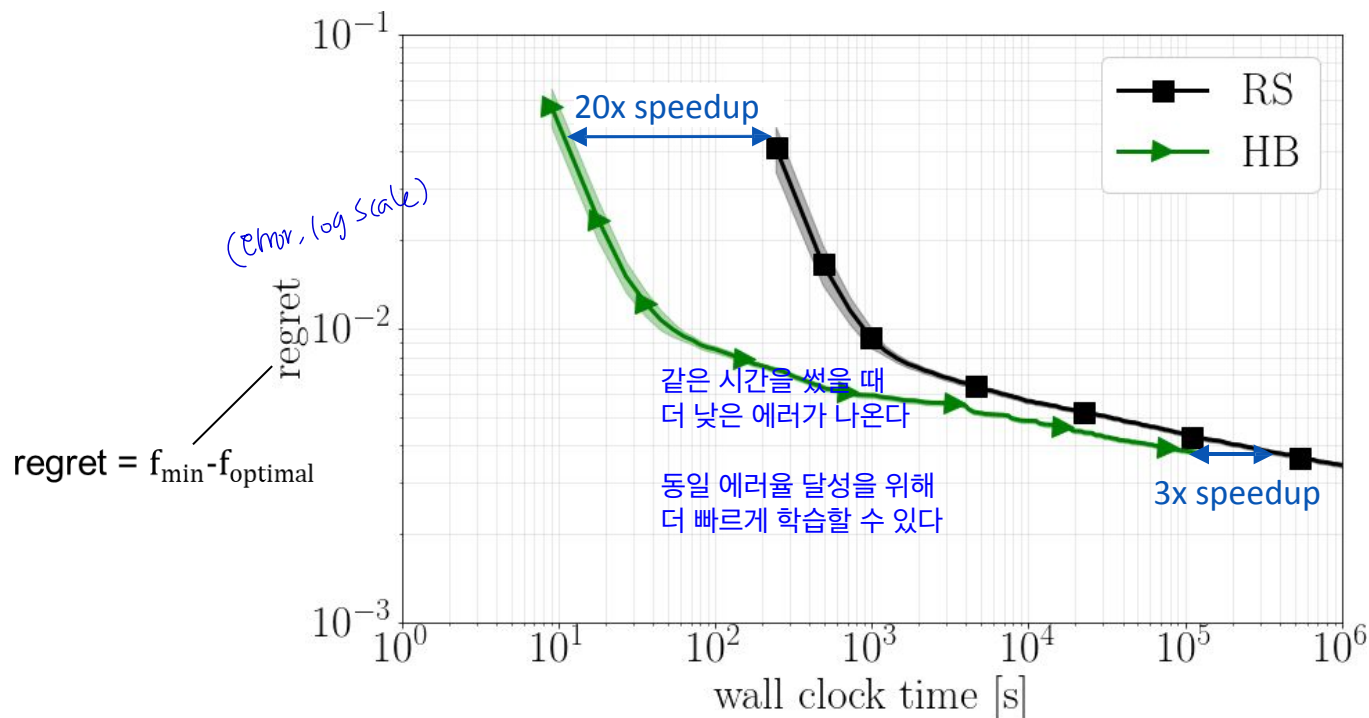
- Strong anytime performance
- Easily parallelizable
- Scalable



- Combining the best of both: BOHB

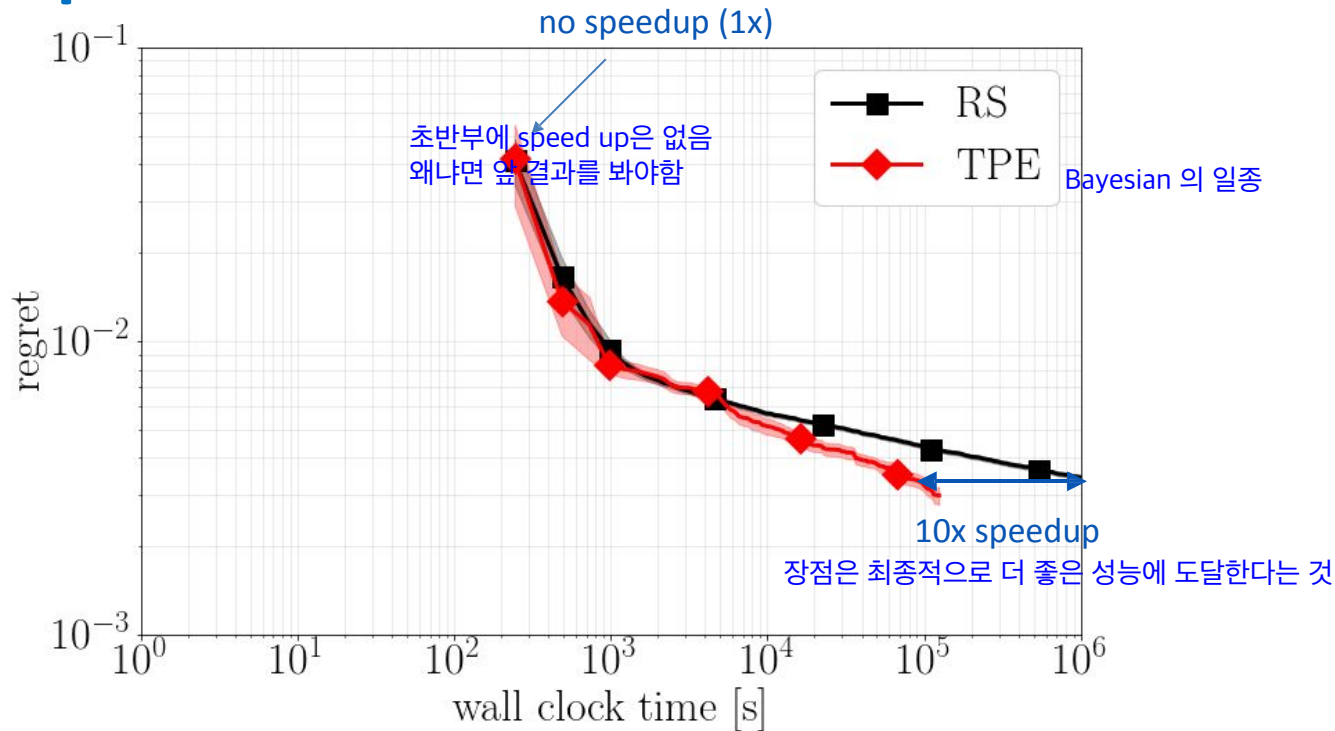
- Bayesian optimization (BO): **choosing the configuration** to evaluate (using TPE)
- HyperBand (HB): deciding **how to allocate budgets**

HyperBand vs Random Search



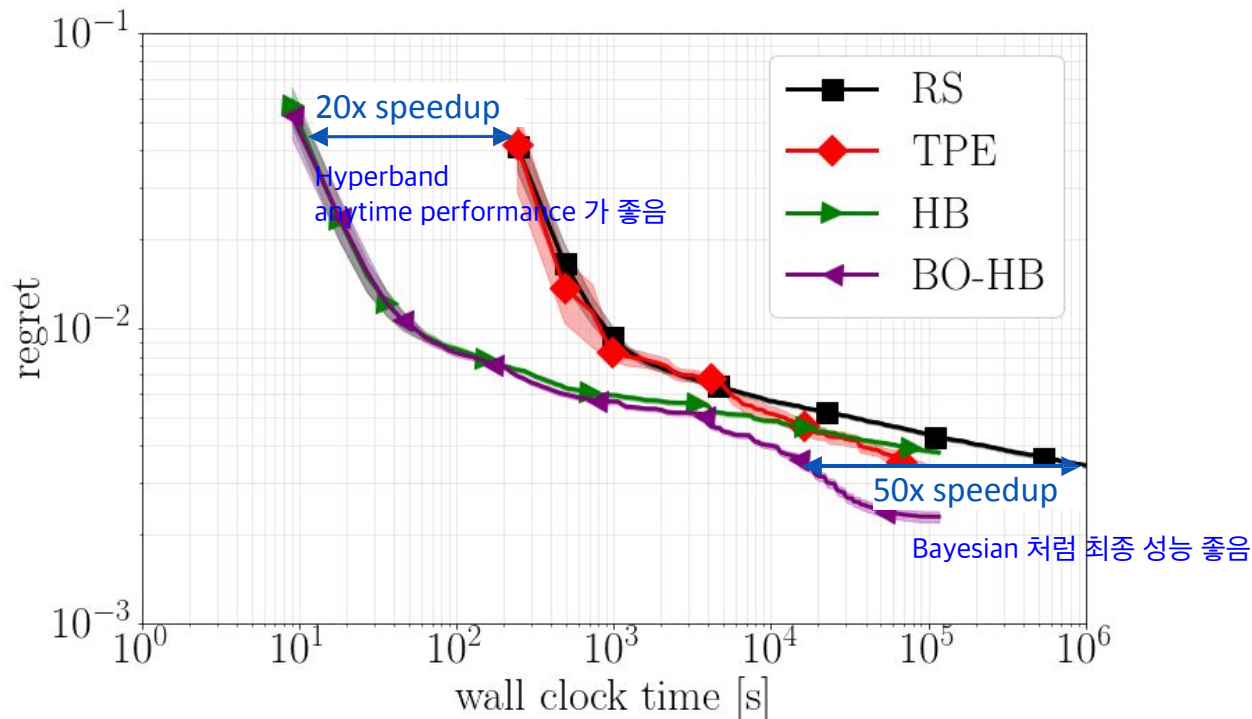
Biggest advantage: much improved **anytime performance**

Bayesian Optimization vs Random Search



Biggest advantage: much improved **final performance**

Combining Bayesian Optimization & HyperBand



Best of both worlds: strong **anytime and final performance**

BOHB Algorithm

대부분의 Auto ML은 이것 이용한다

- Relies on HB to determine number of configurations and budget to evaluate
- Replaces the random selection of configurations at the beginning of each HB iteration by a TPE model-based search
- Update TPE using intermediate losses during SHA, prioritize results from higher budget
 - Use budget $b = \operatorname{argmax} \{ D_b : |D_b| \geq N_{min} + 2 \}$, where D_b is observation with budget b and N_{min} is minimum number of points to build a model

References

- AutoML tutorial at NeurIPS 2018¹⁾
- AutoML-101²⁾
- Scalable Machine Learning and Deep Learning, KTH³⁾
- Automated machine learning⁴⁾

1) <https://www.youtube.com/watch?v=0eBR8a4MQ30>

2) <https://www.automl.org/wp-content/uploads/2021/03/AutoML-101.pdf>

3) https://id2223kth.github.io/slides/2022/11_automl.pdf

4) Hutter, Frank, Lars Kotthoff, and Joaquin Vanschoren. *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.